Similarly, `from` returns the tail from the element at the passed index to the end. If the index is greater than the length of the array, it returns an empty array.

```
%w(a b c d).from(2)  # => %w(c d)
%w(a b c d).from(10) # => []
[].from(0)           # => []
```

The methods `second`, `third`, `fourth`, and `fifth` return the corresponding element (`first` is built-in).

Thanks to social wisdom and positive constructiveness all around, `forty_two` is also available.

```
%w(a b c d).third # => c
%w(a b c d).fifth # => nil
```

Defined in `active_support/core_ext/array/access.rb`.

10.2 Adding Elements

10.2.1 prepend

This method is an alias of `Array#unshift`.

```
%w(a b c d).prepend('e')  # => %w(e a b c d)
[].prepend(10)            # => [10]
```

Defined in `active_support/core_ext/array/prepend_and_append.rb`.

10.2.2 append

This method is an alias of `Array#<<`.

```
%w(a b c d).append('e')  # => %w(a b c d e)
[].append([1,2])         # => [[1,2]]
```

Defined in `active_support/core_ext/array/prepend_and_append.rb`.

10.3 Options Extraction

When the last argument in a method call is a hash, except perhaps for a `&block` argument, Ruby allows you to omit the brackets:

```
User.exists?(email: params[:email])
```

That syntactic sugar is used a lot in Rails to avoid positional arguments where there would be too many, offering instead interfaces that emulate named parameters. In particular it is very idiomatic to use a trailing hash for options.

If a method expects a variable number of arguments and uses `*` in its declaration, however, such an options hash ends up being an item of the array of arguments, where it loses its role.

In those cases, you may give an options hash a distinguished treatment with `extract_options!`. This method checks the type of the last item of an array. If it is a hash it pops it and returns it, otherwise it returns an empty hash.

Let's see for example the definition of the `caches_action` controller macro:

```
def caches_action(*actions)
  return unless cache_configured?
  options = actions.extract_options!
  ...
end
```

This method receives an arbitrary number of action names, and an optional hash of options as last argument. With the call to `extract_options!` you obtain the options hash and remove it from `actions` in a simple and explicit way.

Defined in `active_support/core_ext/array/extract_options.rb`.

10.4 Conversions

10.4.1 to_sentence